

Introduction to AI

Ronald A. Sumida

rsumida@innovationinsoftware.com



Overview

- What is AI?
- Machine Learning
 - Deep learning
 - Models
 - Traditional
 - Generative
 - Agentic
- Large Language Models (LLMs)/GPT
- Agentic AI
- Neural Networks
 - Backpropagation

What is Artificial Intelligence?

- Termed coined by John McCarthy in 1955
- From Microsoft Cloud computing dictionary:
 - "It's the capability of a computer system to mimic human-like cognitive functions such as learning and problem-solving."
 - <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-artificial-intelligence/>
- Narrow (aka Weak AI)
 - Goal: Computer system can outperform human at narrow range of tasks
 - No consciousness or emotion
- Artificial General AI (AGI)
 - Goal: Computer system can outperform human in any intellectual task
- Artificial Super Intelligence (ASI)
 - Goal: Computer system can outperform humans in almost every field
 - Includes fields requiring creativity, wisdom and social skills

Learning

- Machine Learning (ML)
 - Subset of AI focused on process by which computers achieve intelligence
 - Most modern algorithms learn by finding statistical correlations in training data
 - Make predictions based on generalizations from training data
- Deep learning
 - Subset of ML using neural networks
 - Uses multiple (“deep”) layers
 - *Supervised*
 - Trained with input and desired output
 - *Unsupervised*
 - Trained with unlabeled data
 - *Semi-supervised*
 - Trained with combination of small amount of labeled data and large amount of unlabeled data

Machine learning models

- Traditional AI
 - Make predictions based on training data
- Generative AI
 - Creates new content in response to prompt
 - Reactive
- Agentic AI
 - Can autonomously pursue multi-step goal in response to prompt
 - Proactive

From Models->Systems->Agents

- Model era (ML/DL)
 - Predict outputs from data
- Prompt era (LLMs)
 - Generate responses from a single prompt
- System era (RAG + tools)
 - Ground responses in enterprise data + take actions
- Agent era (orchestration)
 - Pursue goals with behavior across multiple steps
- Takeaway: Breakthrough is in building AI systems that can act

Deep Learning

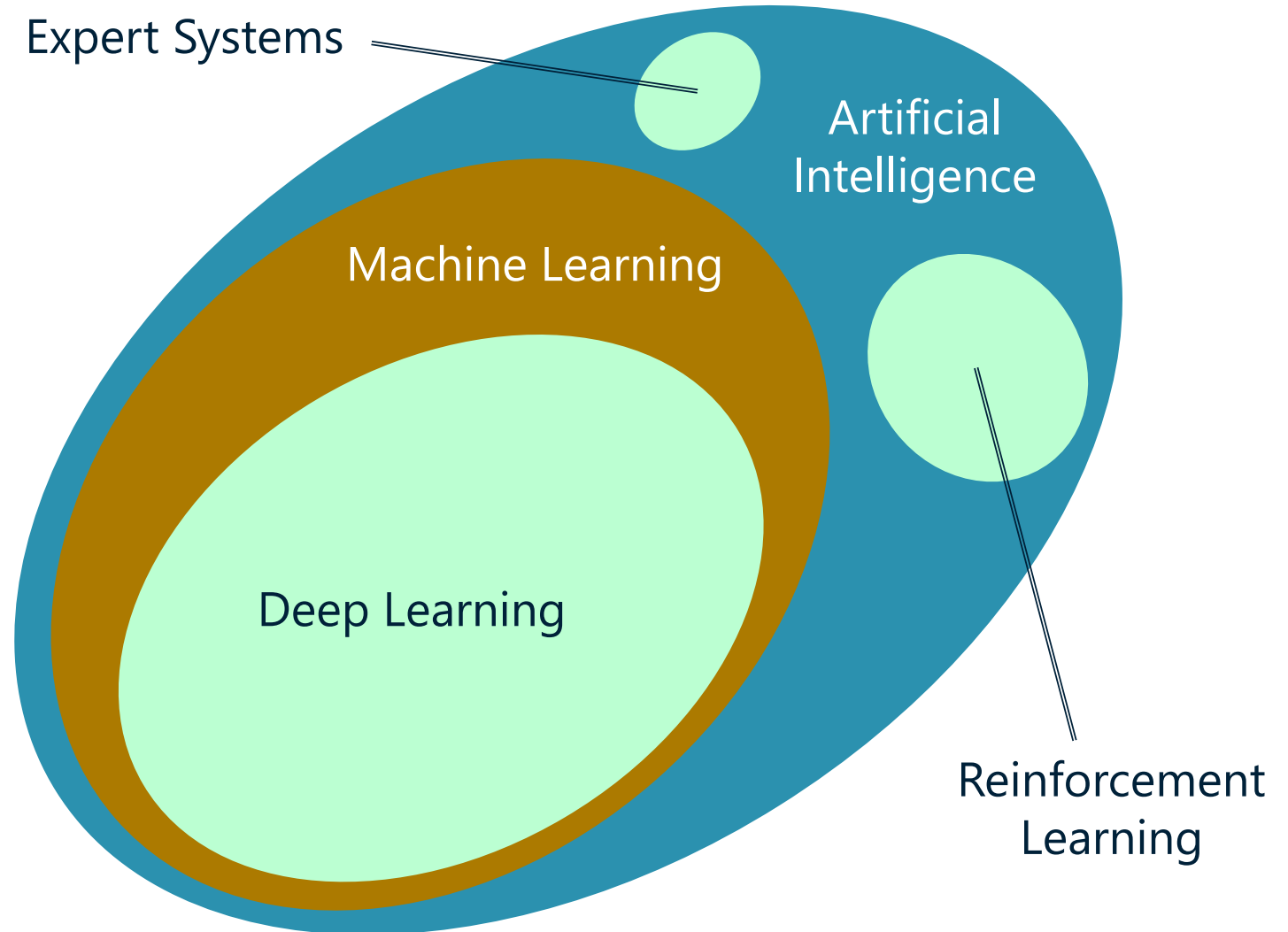
Deep learning is a **superpower**. With it, you can make a computer **see**, synthesize **art**, translate **languages**, render a medical **diagnosis**, or build pieces of a car that can **drive itself**. If that isn't a superpower, I don't know what is.

— **Andrew Ng**, Stanford University

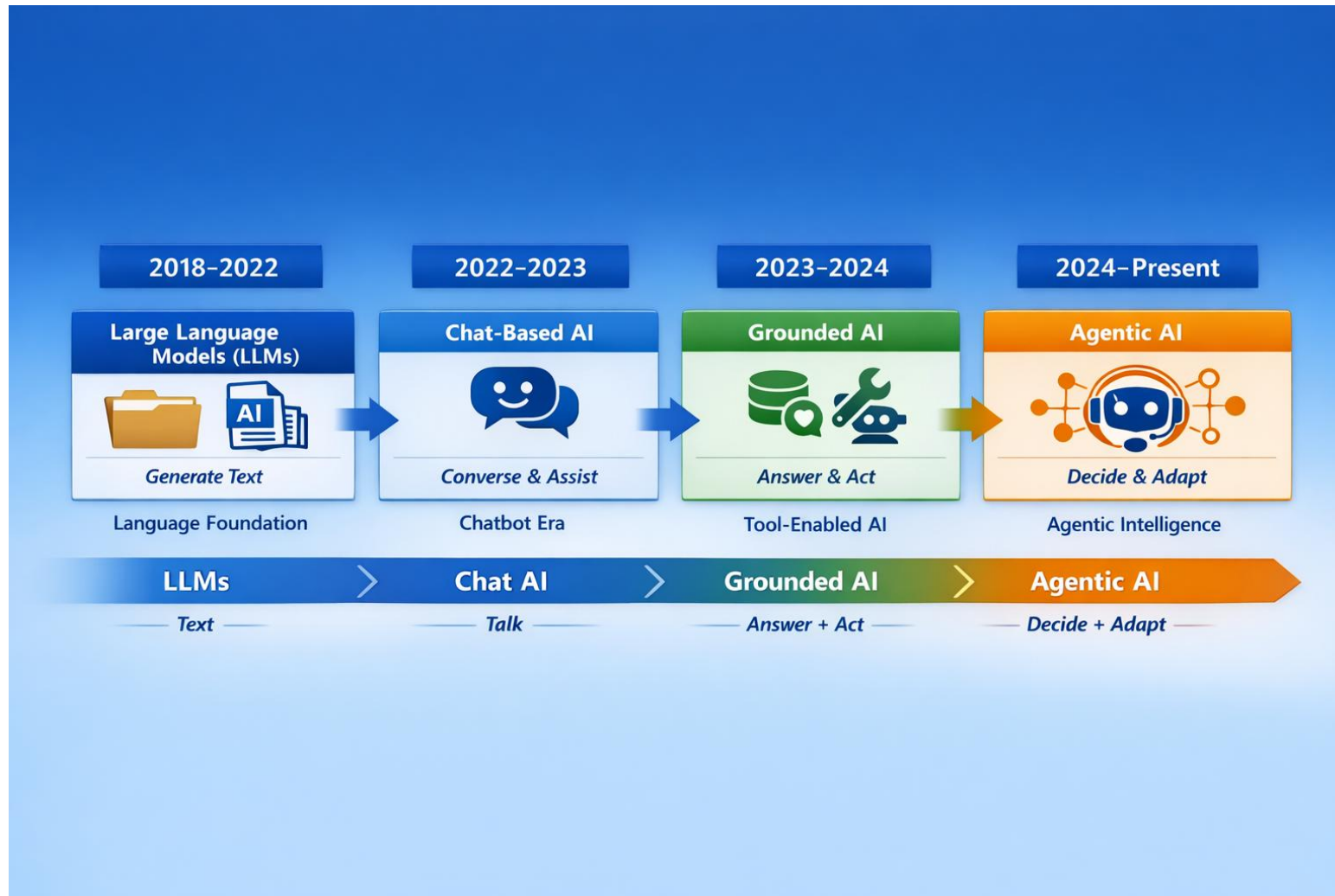


The Big Picture

- 1950s** – Researchers begin pondering whether computers can think like humans and devise **Symbolic AI** – for example, chess programs
- 1960s** – Early **neural networks** are devised, but training is impractical due to lack of computing power and access to compute resources
- 1980s** – Researchers improve **backpropagation algorithms** for training neural networks and apply them to convolutional neural networks
- 1990s** – Researchers at Bell Labs invent the modern **Support Vector Machine (SVM)** algorithm for discovering decision boundaries
- 2000s** – Algorithmic advances in machine learning give rise to **decision trees, random forests, gradient-boosting machines**, and more
- 2010s** – Deep learning explodes due to faster computing hardware (**GPUs**), expanded availability of data, and increased research funding



Modern AI timeline



Generative AI (2018-2023)

- Generates new content
 - E.g., Code/data from GitHub Copilot
- Input is natural language prompt
- Response
 - Based on *single* interaction
 - Constructed from patterns in training data
- Limitation
 - Only provides *static* suggestions

LLMs and GPT (2018-2023)

- Large Language Models (LLMs)
 - Capable of performing variety of NLP tasks
 - Trained on massive corpus of text
 - Using “large” number of parameters
- Generative Pre-trained Transformer (GPT)
 - Type of LLM using transformer architecture (GPT, BERT era)
 - Encoder
 - Generates vector sequence called *embeddings* from input text
 - Decoder
 - Predicts next word from previous words and embeddings
 - Uses *attention* to focus on most significant parts of input and output
 - Limitations
 - No memory, tools, or real-world context on their own

Prompt engineering

- GPT models are prompt-based
 - Input text is entered into prompt
 - Response from model represents completion of input
- Results are very dependent upon prompt
 - Obtaining high quality results can be difficult
 - Requires experience and intuition
- Prompt engineering
 - Construct a *single input prompt* to obtain desired response
 - “More art than science”
 - <https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/prompt-engineering>

Grounded & Tool-Aware AI (2023-2024)

- Answer accurately and take actions
- Retrieval-augmented generation (RAG)
- Access to tools, APIs, and live data
- Constrained by enterprise knowledge
- Limitations
 - Still prompt-driven
 - Limited orchestration and autonomy
 - Only responses
 - No behavior

Agentic AI (2024-Present)

- Next generation AI system capable of
 - *Autonomously* performing tasks
 - Act on behalf of human user or another system
 - *Dynamic* adaption
 - Enabled by reinforcement learning
 - Processing requests in iterative cycles
 - Completing a complex, *multi-step* goal
- Represents an architecture
 - Not a single model

Reinforcement learning (RL)

- Rooted in animal psychology
- Type of ML in which
 - Agent interacts with environment
 - Receives feedback
 - Rewards
 - Penalties
 - Tries to maximize rewards over time
- Enables real-time dynamic adaptation

AI Agents

- Composable software systems that can
 - Make decisions
 - Perform tasks as part of larger workflow
 - Operate autonomously
 - Collaborate with other agents or humans
 - Interact with environment
 - Use multiple data sources for producing generative answers

Context engineering

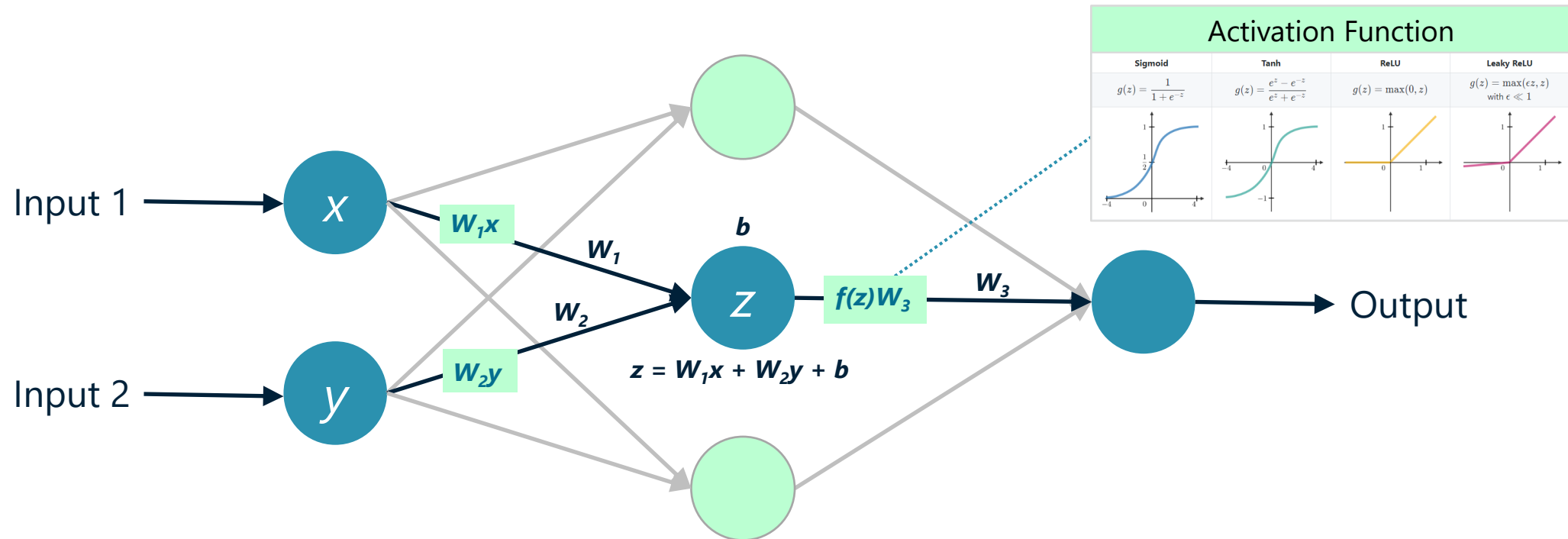
- Construct *entire context* so agent can reason toward a desired outcome
 - Instructions
 - State
 - Memory (short and long term)
 - Dynamic knowledge
 - Tools
- Represents the industry's shift from single-prompts->agent-based systems
- Versus prompt engineering
 - Critical technique in the generative AI era
 - Subset of context engineering
 - Prompt engineering is a tactic
 - Context engineering is an architecture

Context engineering example

- IT support agent
 - Context
 - Role
 - Ticket history
 - Permissions
 - Relevant knowledge bases
 - Available remediation tools
 - Not just a single prompt

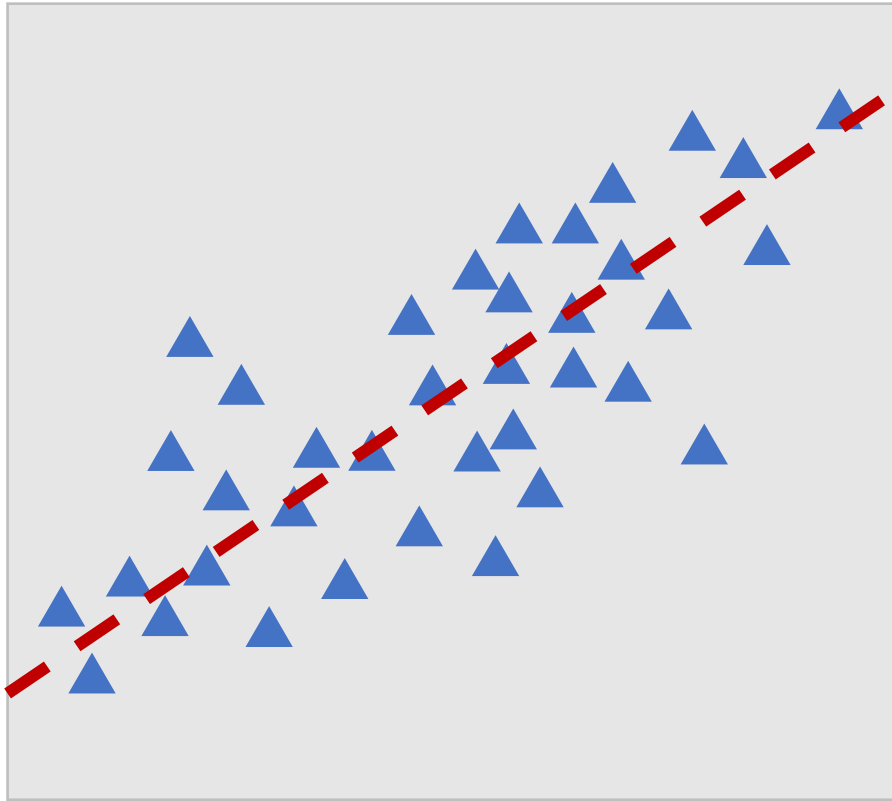
Neural Networks

- Contain layers of *nodes* or *neurons* connected by paths with *weights*
- Data propagates forward by summing products of values and weights, adding biases, and using activation functions to add non-linearity

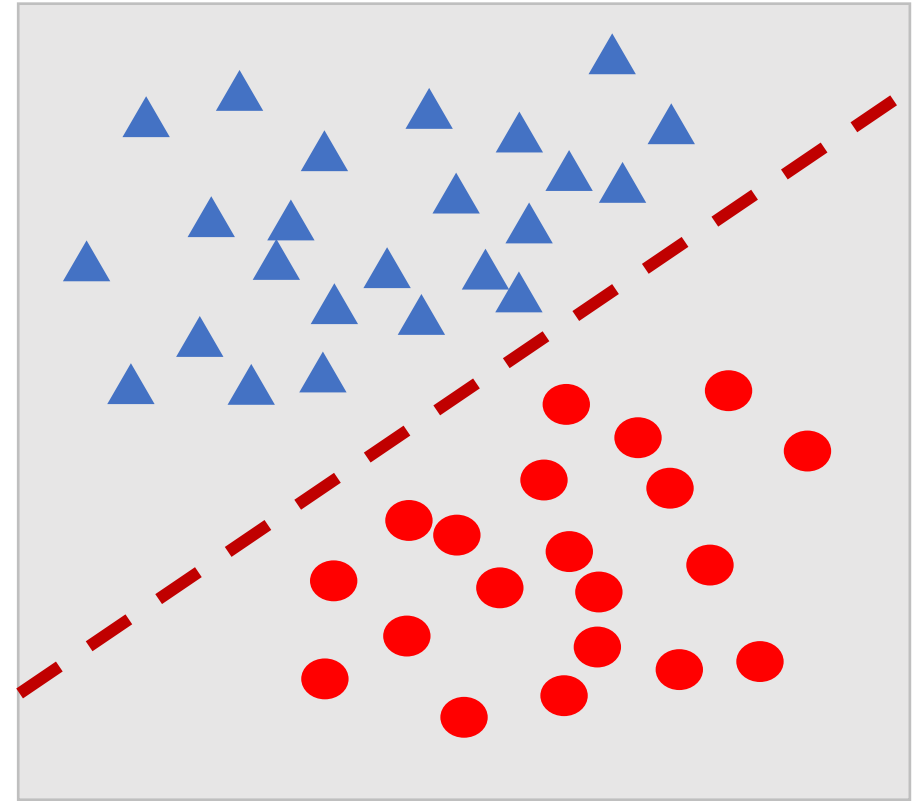


Regression vs. Classification

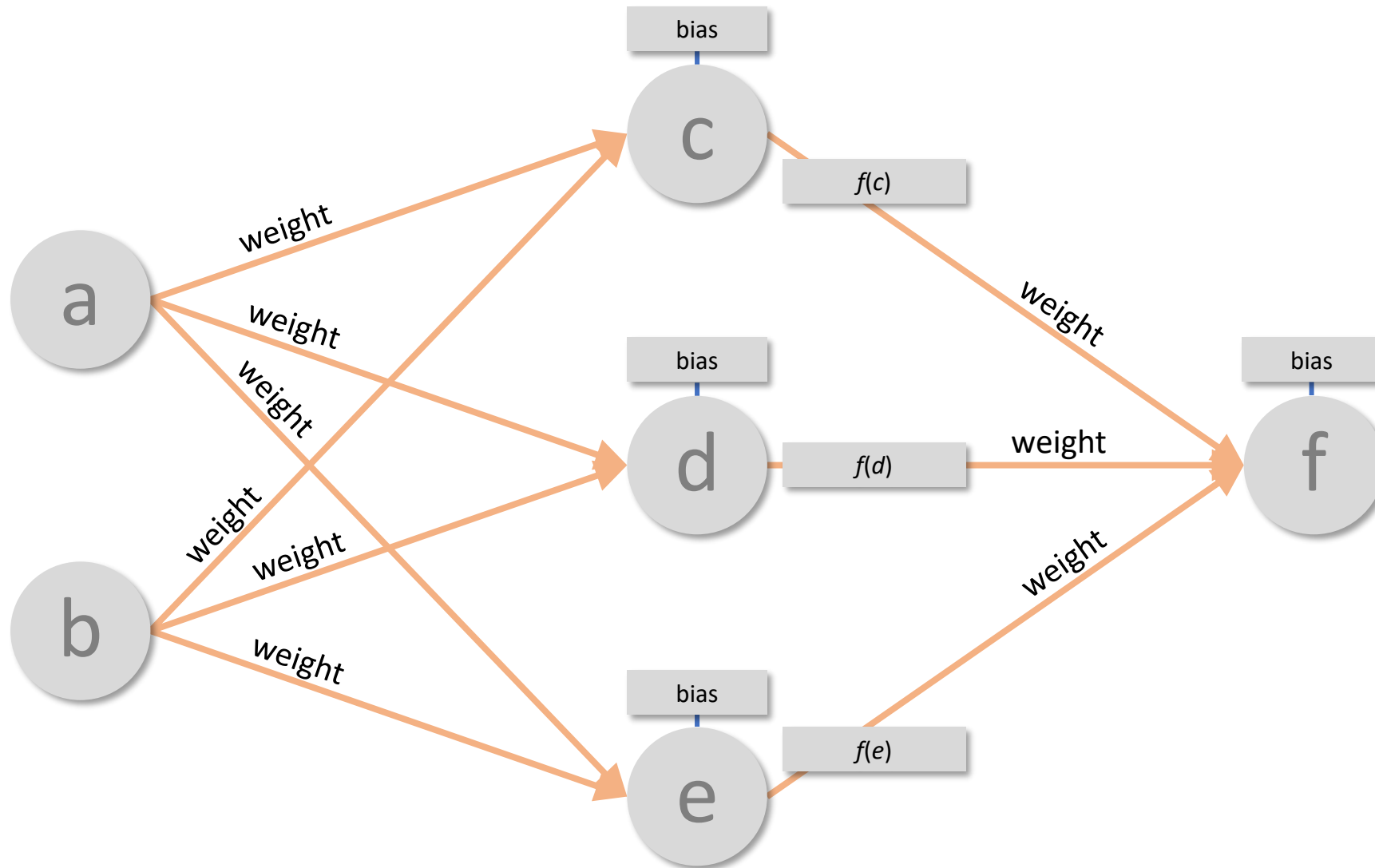
Regression



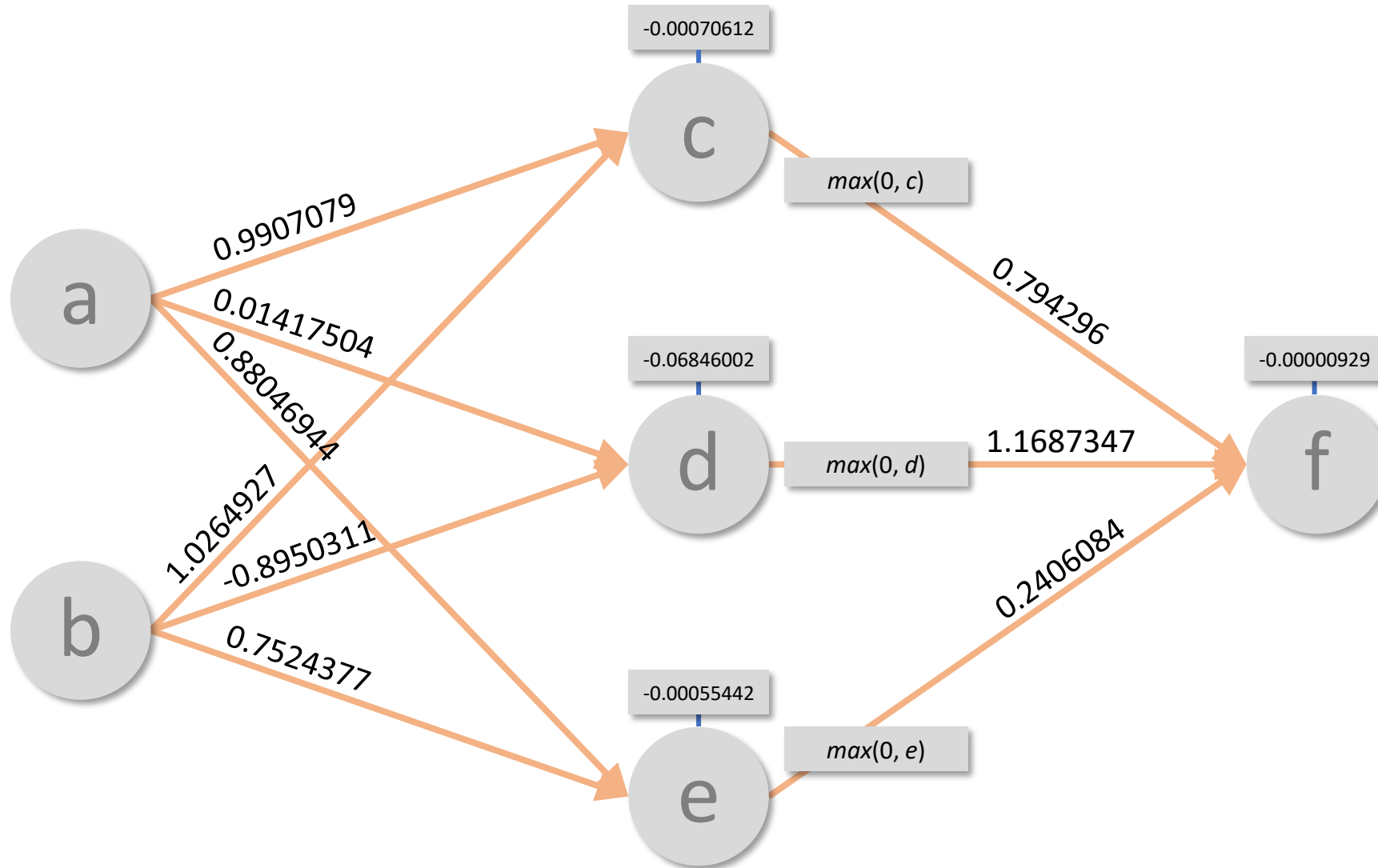
Classification



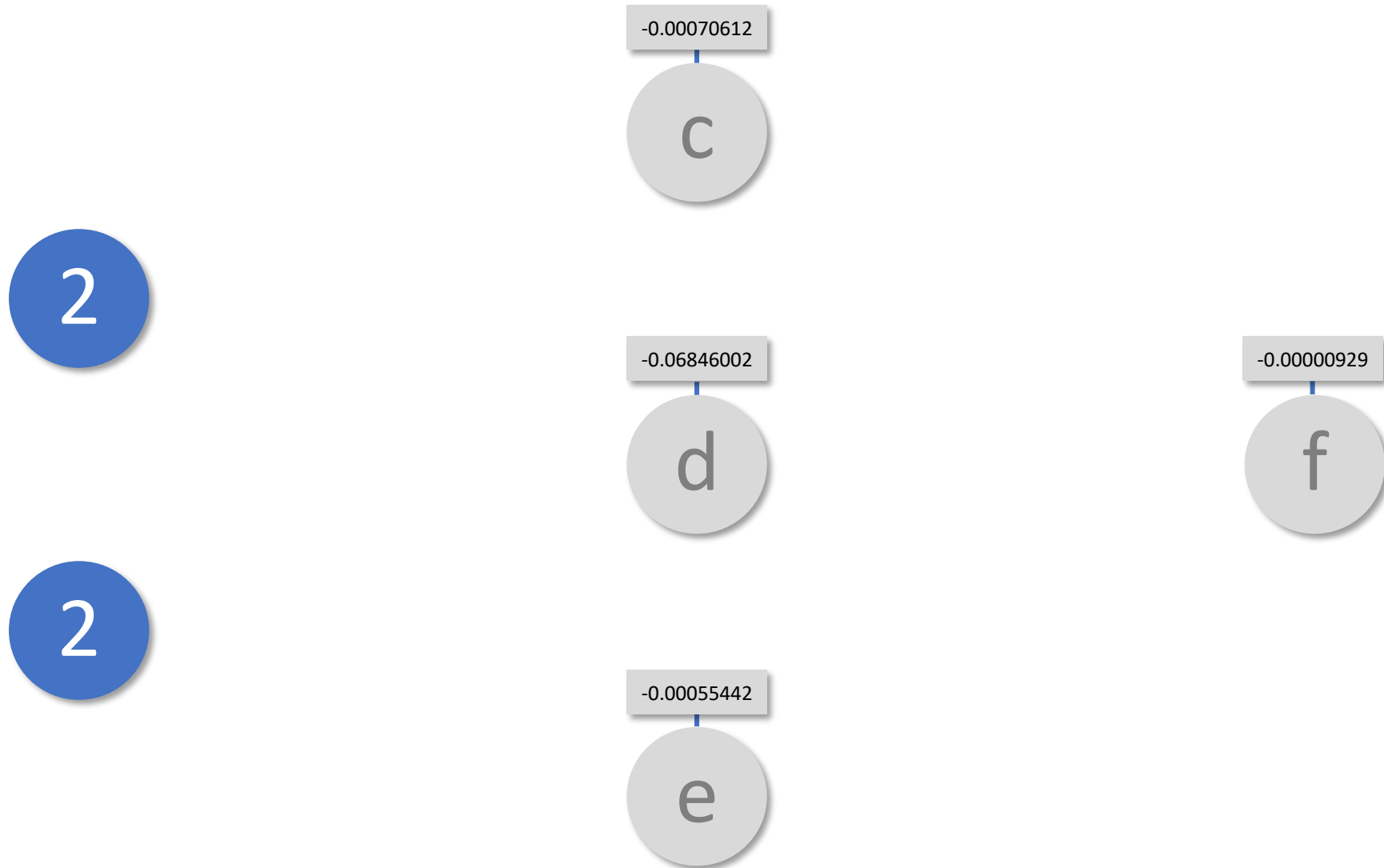
How Neural Networks Work



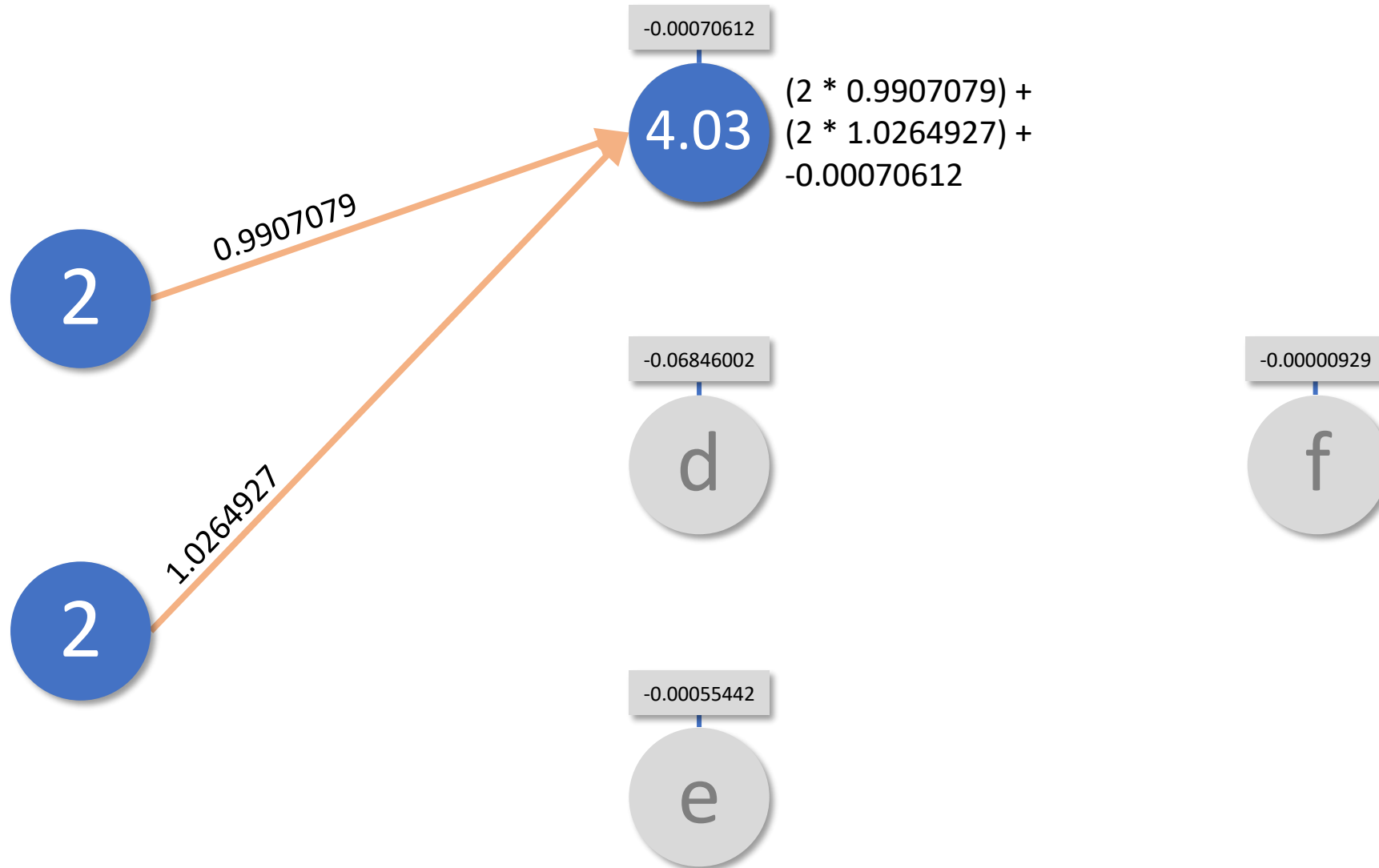
How Neural Networks Work, Cont.



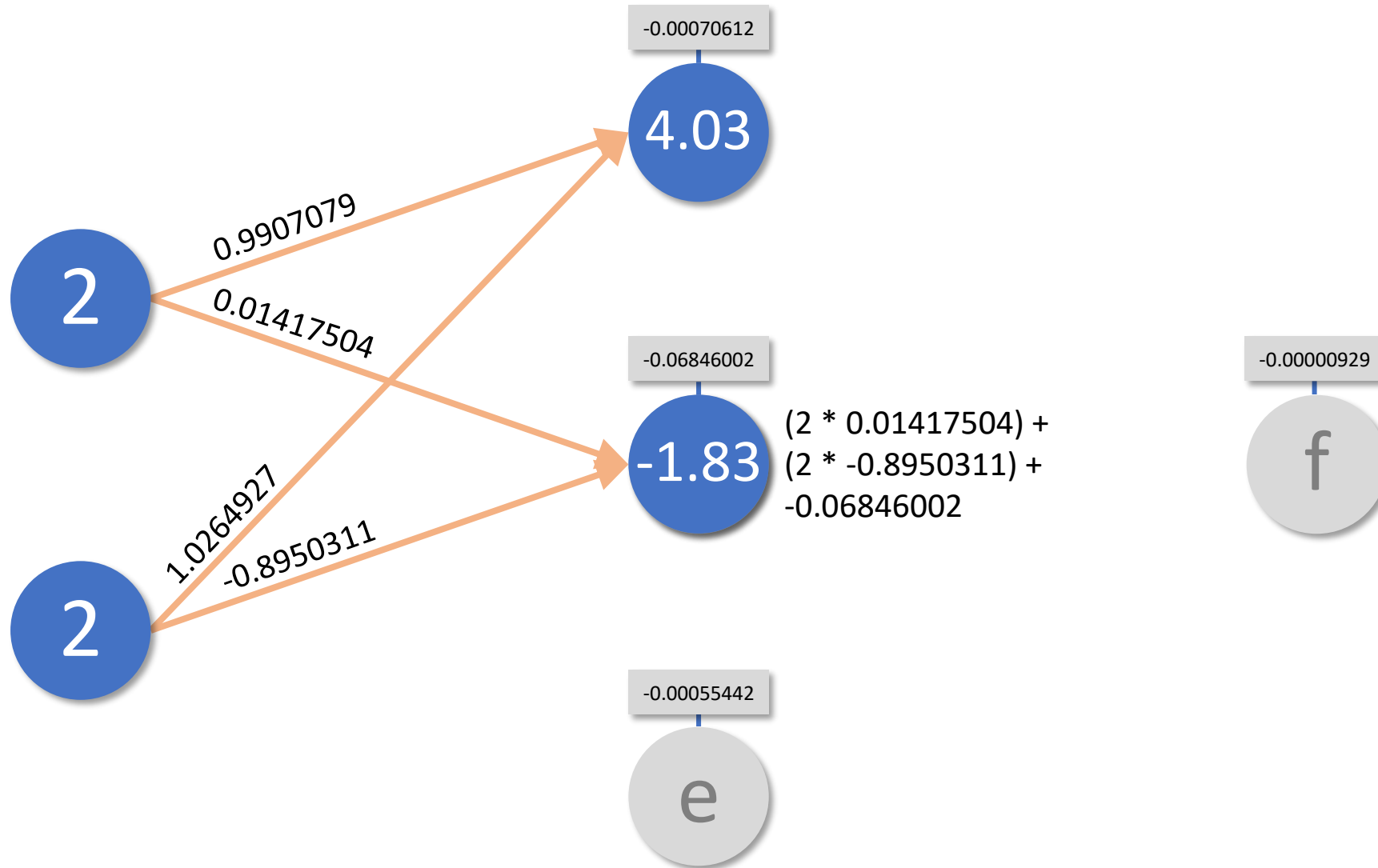
How Neural Networks Work, Cont.



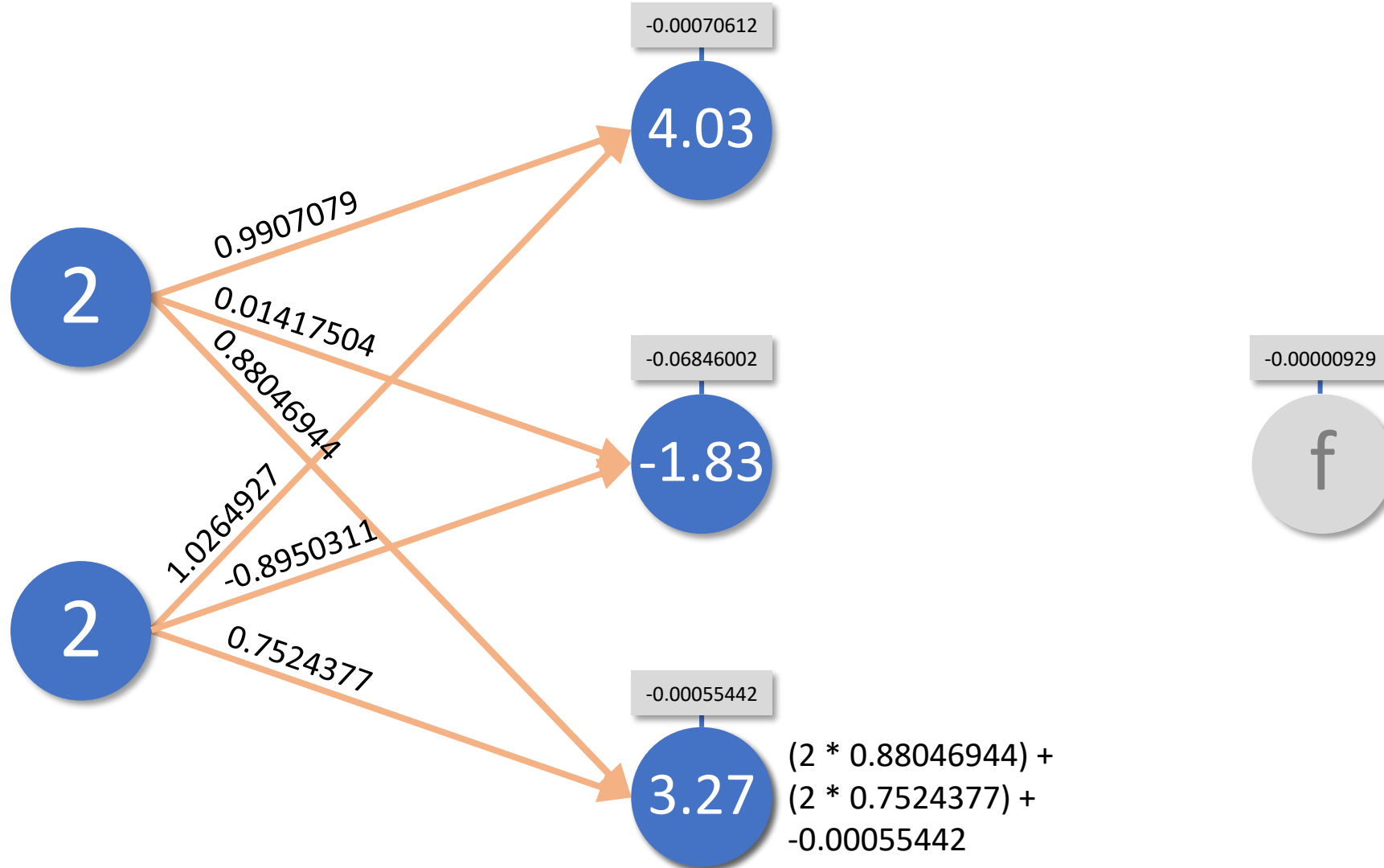
How Neural Networks Work, Cont.



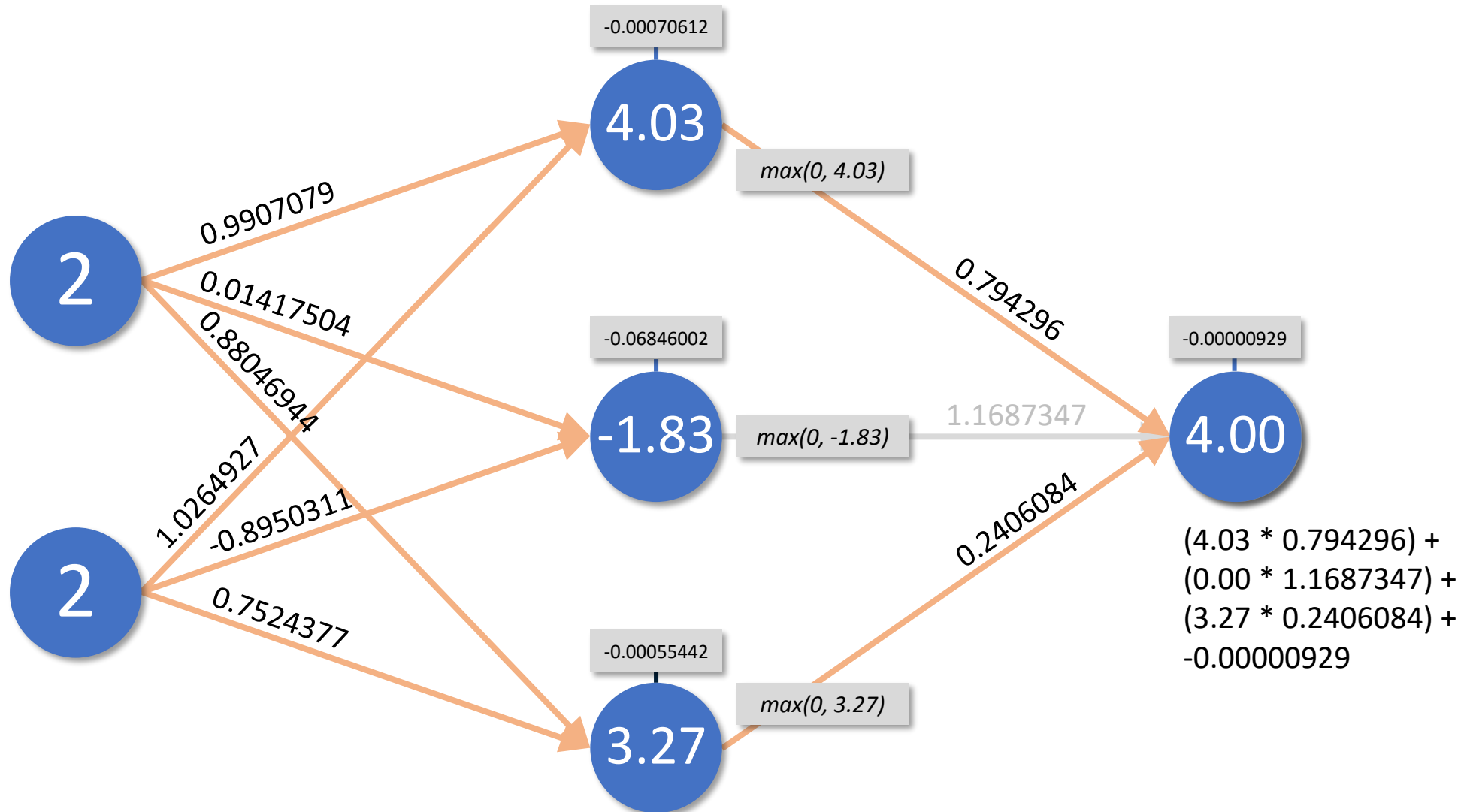
How Neural Networks Work, Cont.



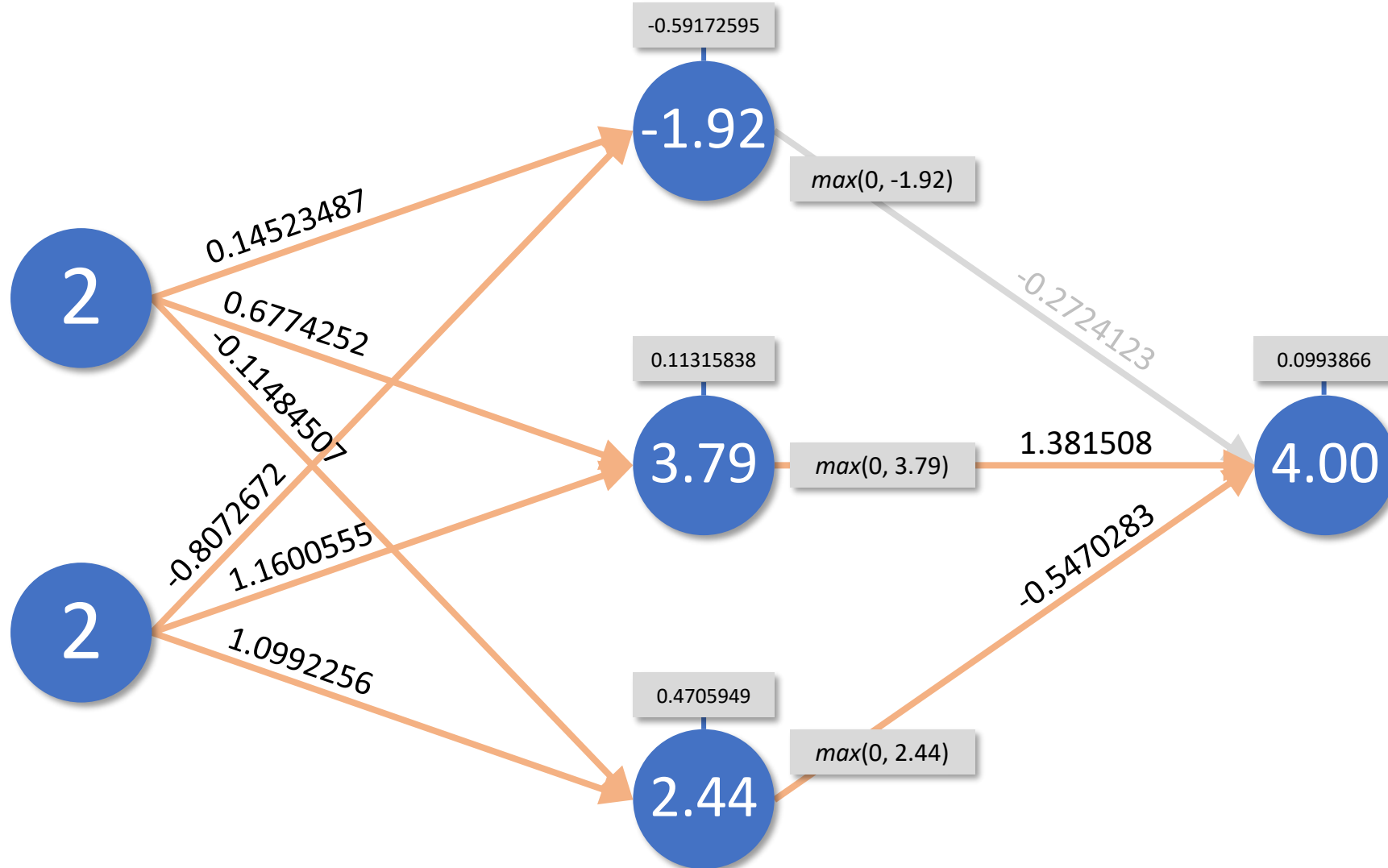
How Neural Networks Work, Cont.



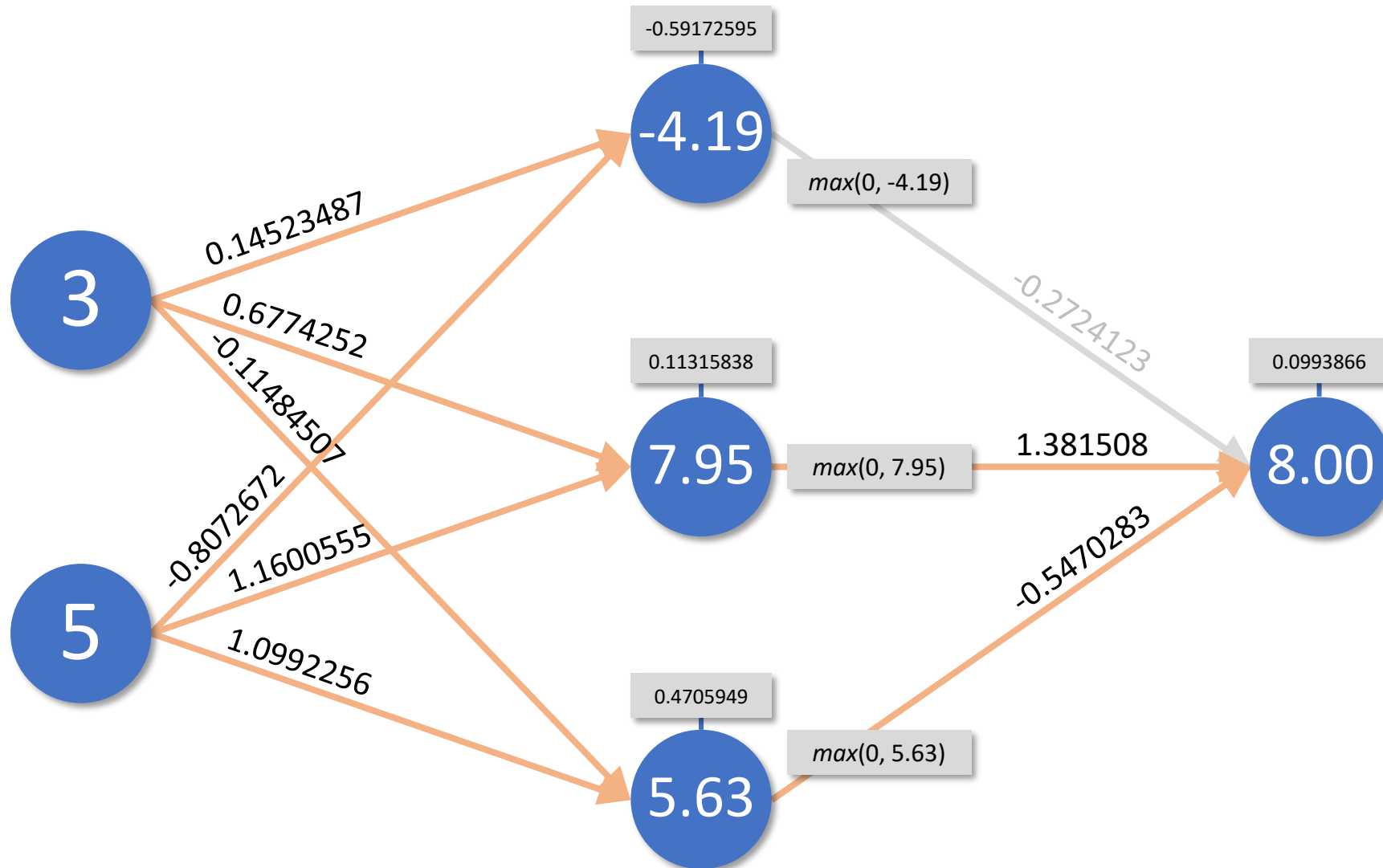
How Neural Networks Work, Cont.



Different Weights and Biases, Same Result



Changing the Inputs

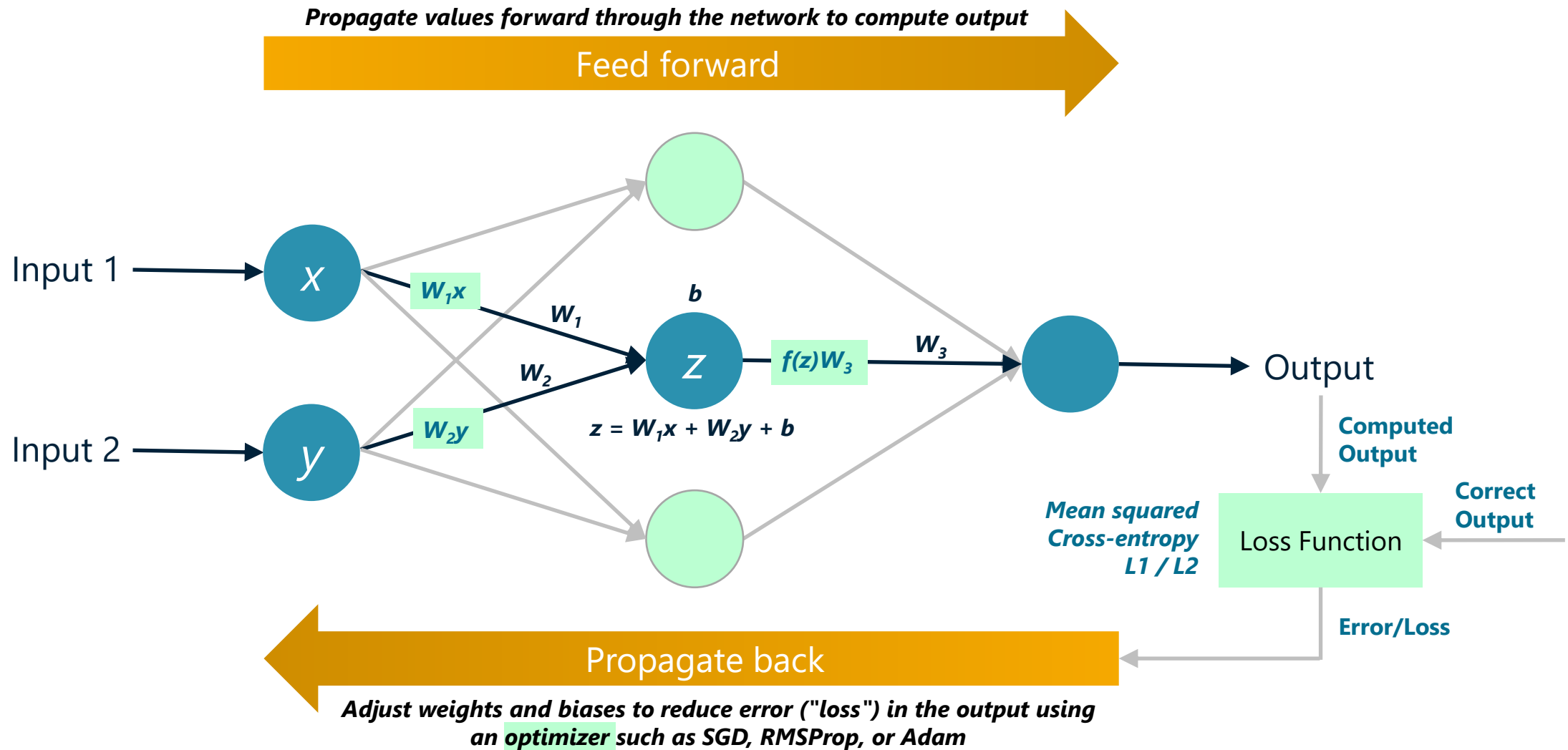


Demo

Multilayer Perceptron

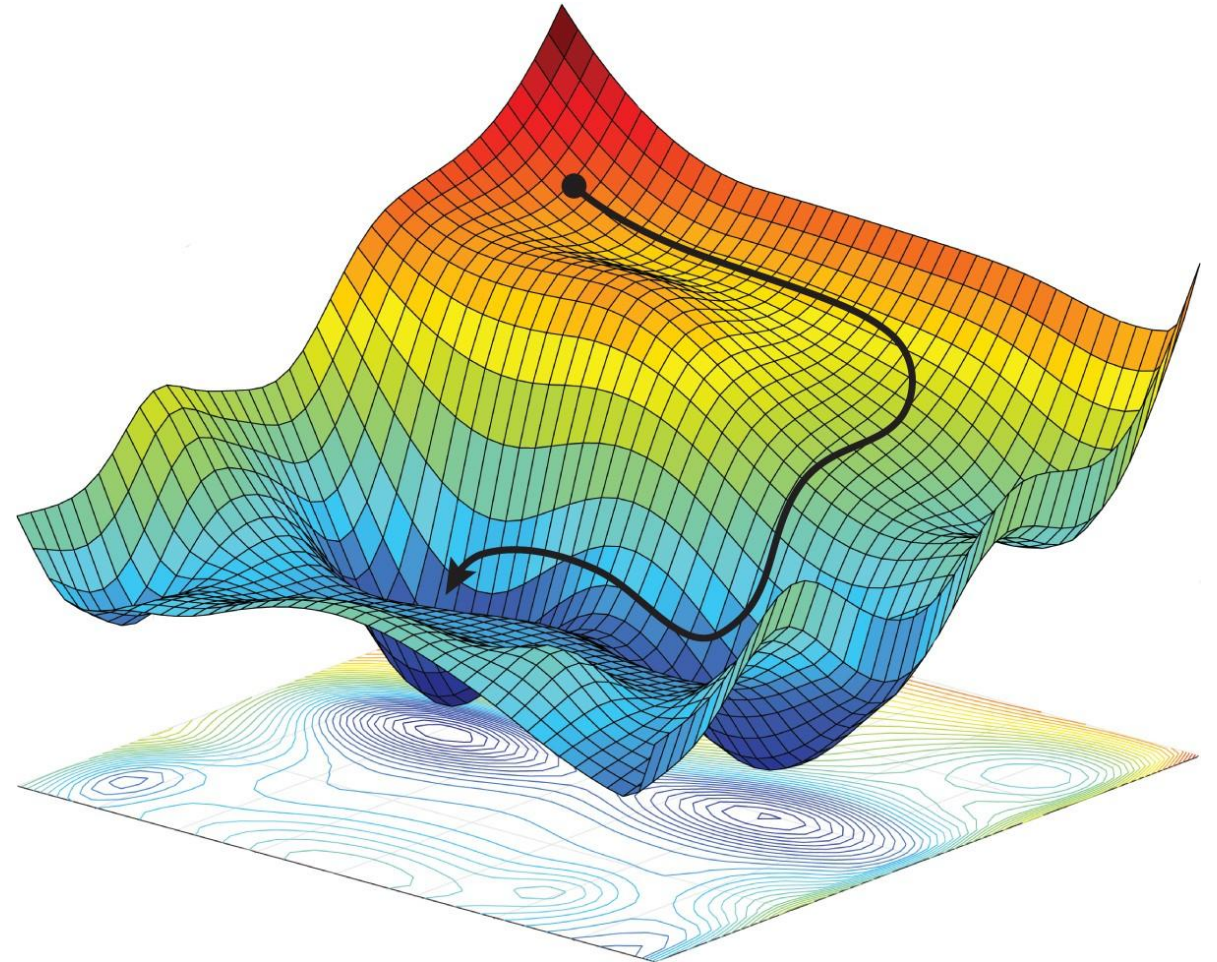


Backpropagation



Gradient Descent

- Training involves navigating multi-dimensional loss landscape searching for global minimum
- Weights and biases are adjusted using gradients computed from partial derivatives
 - Partial derivatives of loss function with respect to individual weights
- Gradients are multiplied by learning rate to make steps incremental



Source: <https://arxiv.org/pdf/1805.04829> ("Spatial Uncertainty Sampling for End-to-End Control")

Updating Weights

$$\nabla = f_{pred} - f_{actual}$$

$$\Delta w_{cf} = c \cdot \nabla \cdot lr$$

$$\Delta w_{df} = d \cdot \nabla \cdot lr$$

$$\Delta w_{ef} = e \cdot \nabla \cdot lr$$

$$\Delta w_{ac} = a \cdot w_{cf} \cdot \nabla \cdot lr$$

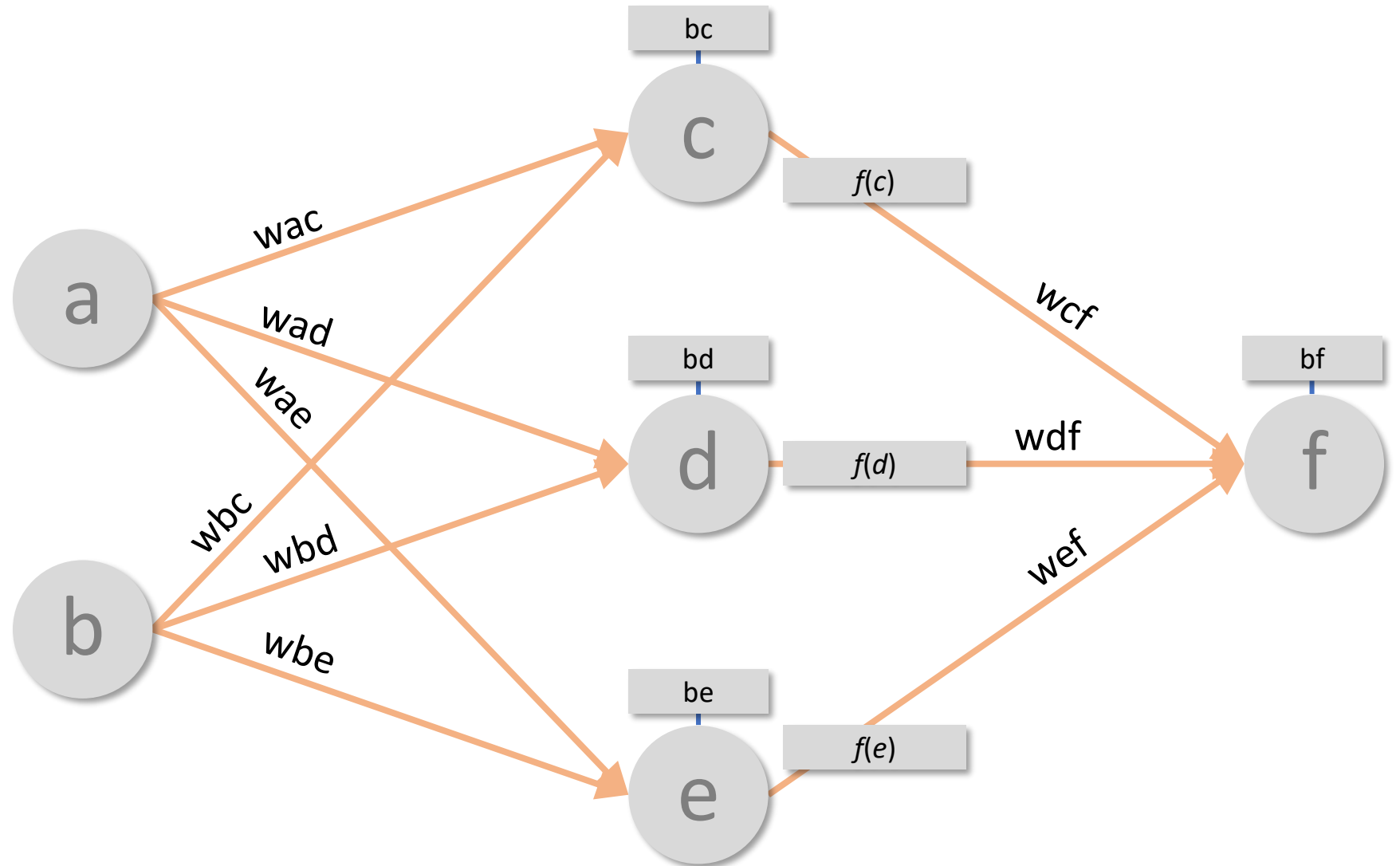
$$\Delta w_{ad} = a \cdot w_{df} \cdot \nabla \cdot lr$$

$$\Delta w_{ae} = a \cdot w_{ef} \cdot \nabla \cdot lr$$

$$\Delta w_{bc} = b \cdot w_{cf} \cdot \nabla \cdot lr$$

$$\Delta w_{bd} = b \cdot w_{df} \cdot \nabla \cdot lr$$

$$\Delta w_{be} = b \cdot w_{ef} \cdot \nabla \cdot lr$$



Updating Weights

$$\nabla = f_{pred} - f_{actual}$$

$$\Delta w_{df} = d \cdot \nabla \cdot lr$$

$$\Delta w_{ef} = e \cdot \nabla \cdot lr$$

$$\Delta w_{bd} = b \cdot w_{df} \cdot \nabla \cdot lr$$

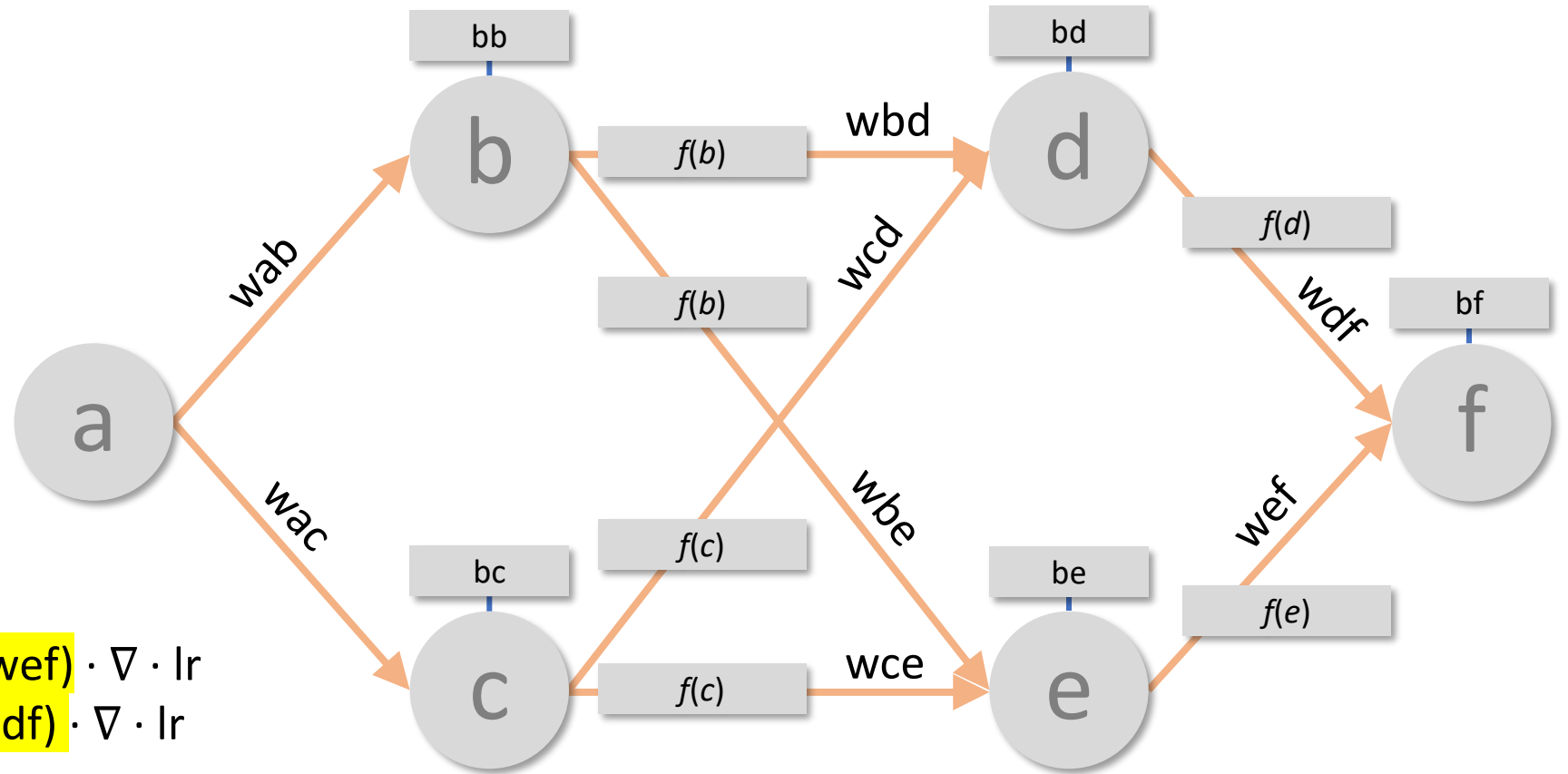
$$\Delta w_{cd} = c \cdot w_{df} \cdot \nabla \cdot lr$$

$$\Delta w_{be} = b \cdot w_{ef} \cdot \nabla \cdot lr$$

$$\Delta w_{ce} = c \cdot w_{ef} \cdot \nabla \cdot lr$$

$$\Delta w_{ab} = a \cdot (w_{bd} \cdot w_{df} + w_{be} \cdot w_{ef}) \cdot \nabla \cdot lr$$

$$\Delta w_{ac} = a \cdot (w_{ce} \cdot w_{ef} + w_{cd} \cdot w_{df}) \cdot \nabla \cdot lr$$

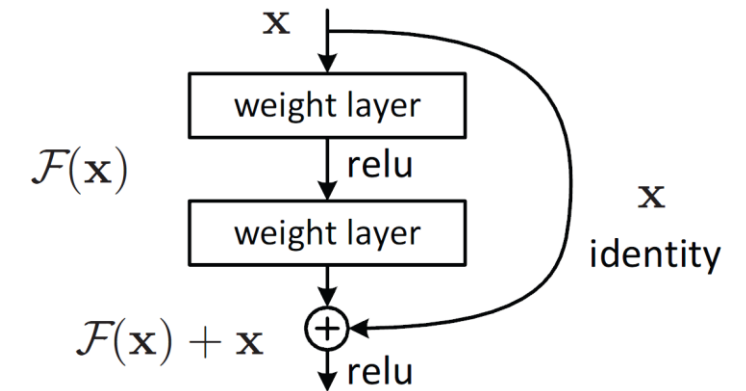


Vanishing Gradients

- Updates get smaller as you propagate backward

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial b} \cdot \frac{\partial b}{\partial c} \dots \frac{\partial z}{\partial w_i}$$

- Three ways to mitigate
 - ReLU – Prevents positive values from getting smaller
 - Normalization – Normalize a layer's output values by subtracting the mean and dividing by one standard deviation (across batch or across features)
 - Residual (skip) connections – Add input to one layer to the output of another
- Makes it difficult to learn long-term dependencies in sequences
 - Memory is not strong for old connections



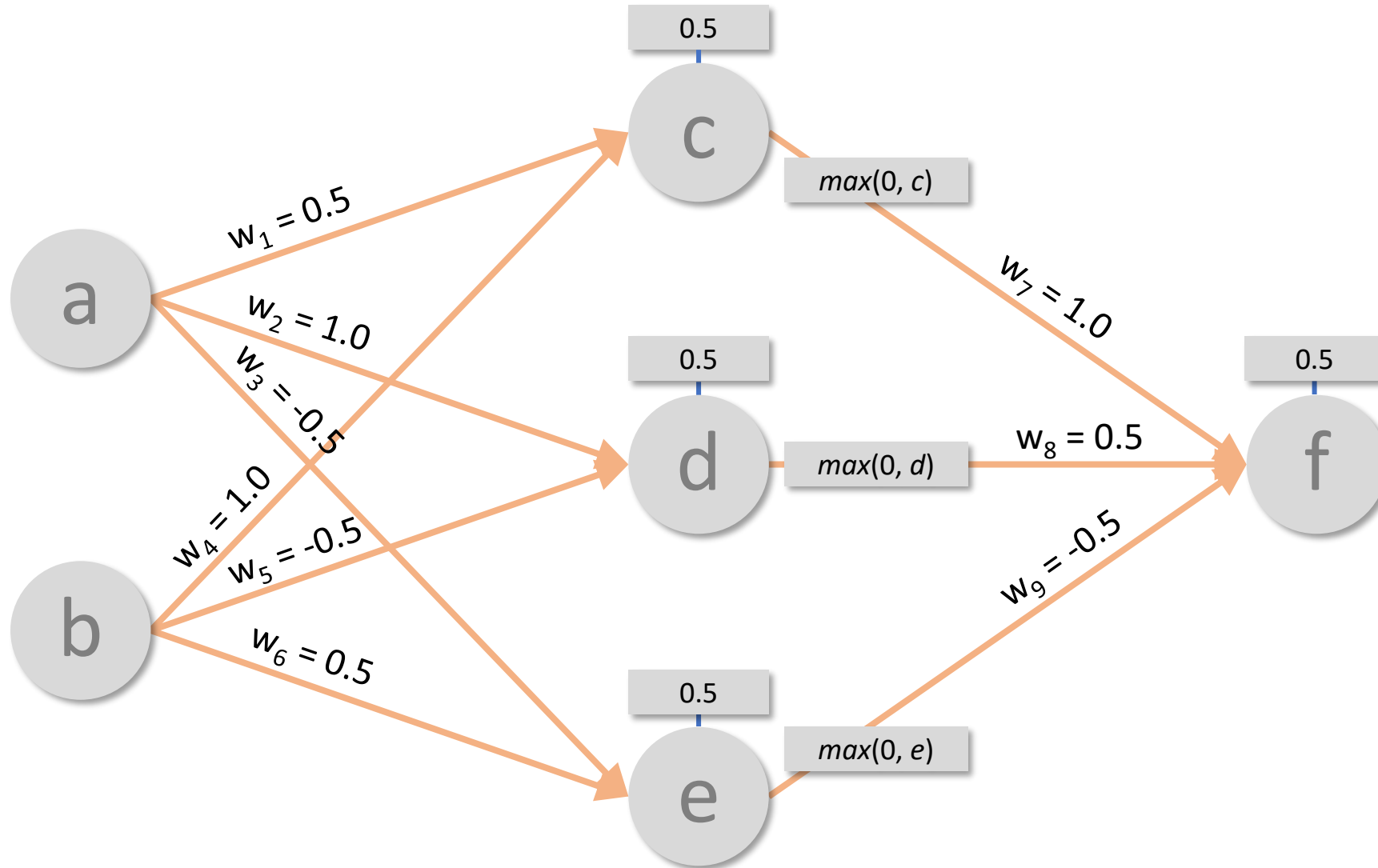
Activation Functions

- Transforms the weighted sum of a node's inputs into an output value
- ReLU
 - Output range: 0 and above
 - Fast and simple
 - Use case: most common choice for hidden layers in deep neural networks
 - Helps reduce the vanishing gradient problem
- Softmax
 - Outputs sum to 1
 - Converts scores into probabilities
 - Use case: multi-class classification
 - Common output layer for classification

Activation Functions

- Sigmoid
 - Output range: 0 to 1
 - Use case: binary classification & probability estimates
 - Can contribute to the vanishing gradient problem
- Tanh
 - Output range: -1 to +1
 - Use case: older hidden-layer activation function
 - Outputs are centered around 0, which often helps training
 - Less common today

Random Weights and Biases



Sizing Neural Networks

- What is the "right" width and depth for a neural network?
 - More neurons provide more learning capacity but increase risk of overfitting
 - Greater width is preferred to greater depth due to vanishing gradients
 - A network with just one or two hidden layers has the capacity to solve even complex non-linear problems
 - 2 x 128 x 1 network features 513 trainable parameters; 2 x 512 x 512 x 1 features 264,705!
 - More neurons == More training time
- Try various widths and depths to find the right balance between training time, accuracy, and the network's ability to generalize

Mini-Batch Gradient Descent

- Backpropagation is expensive
 - Orders of magnitude more time-consuming than a forward pass
- Solution: Do training in batches and perform backpropagation at the end of each batch rather than after every forward pass
 - Run **batch_size** samples through the model and average the error
 - Improves training performance
 - Mitigates effect of outliers in training data
- Proper batch size is determined empirically and depends on both the size and variance of the training data (25 to 50 is typical)

Demo

Backpropagation

